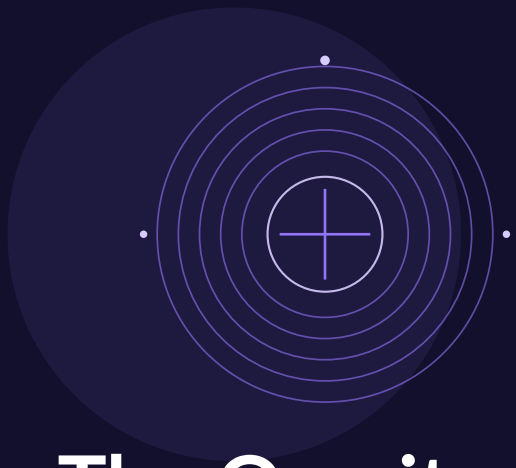
A FIELD GUIDE TO A NEW WAY OF READING



The Omnitome Field Guide

A reader's manual for local-first neural reading

OMNITOME PUBLISHING

First Edition | October 2026

Copyright and Publication

Copyright © October 2026, Omnitome Publishing.

All rights reserved. This publication is an original work prepared for the Omnitome Synthetic Archive. Omnitome is the name of the software project described herein. Product names and marks belonging to other parties remain the property of their respective owners.

This guide documents Omnitome v1.0.2 as represented by the project repository and session records available on 22 October 2026. It is an explanatory companion, not a warranty, service-level agreement, legal opinion, or promise that every interface label will remain unchanged. Software evolves. Where this guide distinguishes an existing v1.0.2 capability from a proposed Store Edition or v1.1 feature, that distinction is intentional.

The user remains responsible for respecting copyright, licence terms, access restrictions, privacy obligations, and the rights of authors and publishers when importing, transforming, retaining, or sharing material. A citation system can help a reader inspect where a generated statement came from; it does not confer permission to copy or redistribute the underlying work.

Omnitome Publishing

First edition, October 2026

Table of Contents

A Note Before You Begin iv

I A Better Shape for Reading 1

1 Why Reading Needs a Lens 2

2 The Two-Representation Invariant 4

3 Local-First, Explained Without Marketing Fog 6

II Getting the First Session Right 9

4 First Launch: Start Small, Learn the Room 10

5 Choose Your Intelligence Provider 12

6 Connect OpenRouter 14

7 Connect a Local LLM with Ollama 16

8 Connect a Local LLM with LM Studio 18

III From Source to Reading 20

9 Import Is a Workflow, Not a File Picker 21

10 The Queue: Watch the Work Happen 24

11 Formats, Extraction, and the Limits of Parsing 27

IV Your Working Library 30

12 The Library: A Shelf That Remembers 31

13 Book Details and the Chapter Map 33

14 Read in Omni, Verify in Research 35

15 Citations, Provenance, and Trust You Can Inspect 37

16 Research: Dossiers, Sources, and Context 40

V Reading With Your Ears and Senses 43

17 Local Neural Narration with Kokoro 44

18 Five Themes, Five Reading Moods 47

19 The Procedural Sound Engine 49

VI Stewardship and Practical Wisdom 51

20 Deletion, Detachment, and the Shape of Ownership 52

21 Privacy, Copyright, and Responsible Use 54

22 Troubleshooting Without Guesswork 56

23 Reading Recipes for Real Life 60

A Quick-Start Checklists 62

B A Compact Glossary 64

C Technical Reference and Official Provider Guides 66

About the Author 67

A Note Before You Begin

Omnitome is best understood as a reading environment with two coordinated jobs. One job is editorial: help you move through long or dense material without requiring you to process every repeated explanation at the same pace. The other is evidentiary: keep the underlying source available so that compression never needs to pretend it is the original document.

That distinction shapes nearly everything you will encounter here: why the Library and Research surfaces are separate, why an Omni sentence can carry a citation, why an imported book may be visible in one place before it is ready in another, why the application has an explicit queue, and why sound design and themes are more than decorative afterthoughts.

This book describes the **Omnitome v1.0.2 Stable** product state recorded in the repository handoff on 22 October 2026. It focuses on the Windows desktop installer and the shared application architecture documented in the public repository. Model providers, operating-system behavior, and the user interface may change over time. Treat exact field names and defaults as a snapshot, and use the live Settings screen as the final authority for the version installed on your machine.

A few future-facing ideas deserve a clean boundary. The project has identified a more guided Welcome Wizard, Synthetic Archive generation and installation, and commercial Store Edition distribution as v1.1 productization priorities. This guide may explain the intended shape of those workflows where that helps orient the reader, but it does not claim that planned Store Edition features are already shipped in v1.0.2. A plan is not a button, and a button is not a verified workflow until it has been exercised in the actual application.

A final vocabulary note: in earlier development stages, the product was called Smart Reader. The current user-facing name is **Omnitome**. Historical project records may use “Smart Reading,” “Omni Reading,” or “Smart Chapter” when describing earlier phases or internal concepts. In this guide, **Omni Reading** means the current derived reading representation, and **Original Reading** means the source-oriented representation available through Research and provenance navigation.

PART I

A Better Shape for Reading

1

Why Reading Needs a Lens

A long book is not difficult only because it is long. It is difficult because not every paragraph carries the same amount of new information. A technical chapter may state a principle, restate it in a diagram, walk through an example, add a caveat, and revisit the same point in a recap. A novel may repeat a character's doubt across several scenes before the plot advances. An essay may circle an argument from multiple angles before reaching its most useful conclusion. That repetition can be deliberate, beautiful, pedagogical, or necessary. It can also make a reader spend the same unit of attention several times on material they already understand.

Ordinary summarization tends to solve the problem by making a much smaller document and accepting substantial loss. It may keep the conclusion but shed the reasoning. It may retain the plot event but flatten the atmosphere around it. It may state what a chapter is "about" without retaining the sequence that made the argument persuasive. That is useful for some tasks, but it is not the same as reading.

Omnitome approaches the problem as **semantic compression rather than a generic summary**. Its intent is to create a denser reading path that preserves meaningful concepts, causal links, narrative developments, qualifications, and technical distinctions while reducing redundancy. The result is not a replacement for the source. It is a reading lens: a more compact route through the material, with doors back to the uncompressed evidence.

The distinction matters most when the source has real structure. A textbook is more than a bag of facts; the order in which ideas are introduced affects whether a learner can follow them. A research paper is more than its abstract; the methods, limitations, and evidence are part of its meaning. A story is more than its plot points; relationships, reveals, and local consequences give those points their force. Omnitome's editorial pipeline therefore tries to preserve structure rather than reduce every source to one undifferentiated block of prose.

The application has a staged import and synthesis pipeline. Broadly, it extracts usable text, prepares a semantic index, classifies the source, plans an editorial outline, generates a synopsis, synthesizes chapter-sized reading units, and attributes those units back to source passages. Different document types and sources can exercise different branches of that pipeline; the stages are not simply a progress animation. They are separate kinds of work, and their separation makes failures easier to locate.

Omnitome's synthesis contract aims for compact chapter units, generally around **250–360 words**, with a documented strict ceiling of 500 words. That target is a

pacing design, not a universal law that every source can be compressed into the same ratio without consequence. The project history contains both aspirational compression language and empirical examples with different ratios. For that reason, this field guide does not promise a fixed "seven-to-one" outcome for every document. The dependable principle is the target reading shape and the effort to bound information loss, not a magic ratio guaranteed for every book.

There is another important consequence: a compressed sentence can sound confident while still being incomplete or wrong. Model output is not made true by being fluent. Omnitome's response is to make source inspection part of the reading experience. The generated representation should help you move quickly; citations should help you decide when to slow down, examine the evidence, and judge the statement for yourself.

Think of the application as a desk with two materials laid out at once. The first is the source: the book, article, paper, or text you brought in. The second is a new, shorter reading surface built from it. The shorter one helps you travel. The first one tells you what the journey was built on. You can use the lens for a first pass, then return to the source for a difficult derivation, an ambiguous claim, an exact quotation, or a plot detail that deserves the full scene.

That is the basic bargain. Omnitome asks the model to help spend your attention more intelligently, not to take away your right to inspect what it says.

2

The Two-Representation Invariant

The most important concept in Omnitome is the **Two-Representation Invariant**. It is an architectural rule, a product-design principle, and a reader-protection mechanism.

The two representations are:

1. **Original Reading**, associated with the Research view. It is the source-oriented record of the work you imported, kept separate from the generated reading text.
2. **Omni Reading**, associated with the Library. It is a derived, condensed reading surface created from the source and connected to it through provenance links.

The two are related, but they are not interchangeable. The source is not an editable draft of the synthesis. The synthesis is not a disguised copy of the source. The relationship between them must remain legible.

The team captured this principle in three short design laws.

2.1 Law 1: Source is paper

The original reading is presented as the stable reference. Its visual language should feel neutral, calm, and editorial, not like a special-effects version of the AI output. More importantly, Omnitome does not intentionally rewrite or truncate the imported source to make the compressed representation. The source remains the reference against which the generated text can be checked.

There is a practical nuance here. A PDF, EPUB, DOCX, or web page may be parsed into readable text blocks before it can participate in semantic processing. That extraction can normalize formatting, identify headings, or fail on difficult layouts. The source-oriented view is therefore not a guarantee of a pixel-for-pixel reproduction of the original page. Preserve the original file when its visual layout, tables, images, page numbers, or typography matter to your work. Use the Research text and its highlighted passages to inspect the extracted content, and consult the original file itself for layout-specific questions.

2.2 Law 2: The lens is tinted

Omni Reading has its own visual identity. It presents the compact synthesis as a distinct reading experience rather than pretending that it is the author's exact prose. Its chapters are editorially structured and may use theme-aware styling.

This “tint” communicates a useful fact before you read a single sentence: this is a derived representation.

The lens is not a claim that the model knows better than the author. It is a tool for seeing the source at a different density. It may bring the argument’s structure into focus, but it can also omit a detail that becomes important to your particular purpose. The correct response is not to abandon compression; it is to keep the ability to change scale.

2.3 Law 3: Derivation never masquerades as source

A synthesized statement should provide a deterministic path back to supporting source material. In the reader, provenance is not meant to live only in backend records or a diagnostic log. It appears where a reader needs it: alongside the derived sentence, available to open, inspect, and use to navigate back to the relevant passage.

These laws reinforce each other. If the source is editable by the synthesis process, there is no dependable reference. If the lens looks exactly like source text, users can confuse generated prose with the author’s words. If a compressed claim cannot be traced, the user has to trust the model’s tone instead of examining its basis.

2.4 What the invariant does and does not mean

The invariant does mean that source material and derived output have distinct roles, that the source should remain available as evidence, and that provenance belongs in the reader-facing experience. It also informs lifecycle behavior. A book may be present in both Library and Research; it may remain only in Research after its Omni reading is removed; or an Omni reading may remain after the large source file is detached, with an explicit status indicating that the source is no longer attached.

It does **not** mean that every imported file can be decoded perfectly, that every source sentence is semantically unambiguous, that model-generated claims are guaranteed correct, or that citation presence grants permission to reproduce copyrighted material. Provenance makes verification possible and practical. It does not replace judgment.

It also does not require every reading session to bounce constantly between views. For everyday progress, start in Omni Reading and let the text carry you. Open the source when the stakes of precision rise: a number you intend to cite, a theorem step you need to understand, a character’s motive that seems misrepresented, a surprising assertion, or a sentence whose wording matters legally or academically.

The invariant can be summarized in a single sentence: **Omni helps you read; Research lets you verify.** Protecting that division keeps the product honest and gives the reader control over how much compression is appropriate at each moment.

3

Local-First, Explained Without Marketing Fog

"Local-first" is a description of where an application keeps its center of gravity. It is not a synonym for "nothing ever leaves the machine." Omnitome is designed as a local-first desktop application: the user interface, application data, import queue, semantic processing components, vector search, and speech synthesis are coordinated on the user's own computer. The native Tauri shell supervises the application processes. The Node.js backend owns the database writes. A Python sidecar handles neural and document-processing workloads. A user can configure a local language model instead of a hosted generation provider.

That design has practical benefits. Your personal reading library does not need to be a cloud account's central database. Local embeddings and local search can run without sending each query to a remote vector service. Kokoro speech synthesis can operate offline once its required models and bundled runtime assets are present. The application can preserve source records and generated reading units in local storage. Local execution also gives the user a clearer place to reason about disk use, caches, process activity, and backups.

But model choice matters. Omnitome supports both a local OpenAI-compatible LLM endpoint and OpenRouter. When OpenRouter is selected for synthesis, the relevant prompt and source excerpts needed for that request may be sent to the provider. Some workflows may include contextual text, chapter excerpts, or other source-derived material. A local-first application can therefore offer a hosted option without every operation being local. The moment you choose a remote provider, you must think about that provider's retention policy, terms, billing, location, and the sensitivity of the text you send.

3.1 Four layers to keep separate

It is helpful to distinguish four things that are often collapsed into the phrase "AI runs locally."

The application shell is the native desktop container. In v1.0.2, Omnitome uses Tauri v2 with a Rust supervisor. Its responsibilities include window lifecycle and child-process management.

The application database and orchestration are handled by the Node.js backend. Node is the sole SQLite writer, while the frontend and Python sidecar interact through application services instead of writing independently into the database.

This boundary reduces the risk of multiple processes racing to update the same records.

The local neural services include BGE-M3 embeddings, BGE Reranker v2, PaddleOCR for scan recovery, and Kokoro-82M for text-to-speech. These workloads are packaged or managed as local resources by the app's resource and sidecar systems. Some models are loaded lazily because loading every neural component at startup can overwhelm limited RAM or video memory.

The generation provider is the separate decision about where the language model runs. You can point Omnitome at a local endpoint exposed by Ollama or LM Studio, or configure OpenRouter as the remote provider. The local vectorization and speech components do not turn a selected cloud LLM into a local one.

3.2 Offline is a state, not a slogan

A local component can still need its model files before it can run. A local service may also need to be started, a model downloaded, or the first warm-up completed. Omnitome's first-run resource manager exists because the application code and model weights are different things. A working executable does not imply that every large model archive is already installed.

The v1.0.2 packaging work specifically targeted offline behavior for Kokoro's phonemizer resources and spaCy's English model. Those runtime dependencies were bundled to avoid hidden attempts to download packages during speech initialization. The source records also describe local voice resolution and offline testing with Hugging Face hub offline mode. This is the right type of evidence for an offline claim: not a badge alone, but a test with network-dependent paths disabled.

For you, the operational lesson is simple. Decide where you want language generation to happen. Install or download the relevant local model if you choose local inference. Keep internet access available when using a hosted provider or when obtaining missing resources. Verify the selected connection inside Settings. If you need truly disconnected use, test the complete workflow with the network disabled after all resources have been installed; do not infer full offline readiness from the fact that the app launches.

3.3 Local-first does not mean hardware-free

The application can use CUDA acceleration, but a GPU is not mandatory for every task. The project includes guards and fallback paths for constrained systems. Embedding and reranking can still take time when hardware memory is limited, and local LLM generation can be slow on a small machine. A local model that fits into memory but responds poorly to long-context synthesis may be a worse choice than a more modest model with reliable instruction following. A hosted provider may be more convenient for certain heavy tasks, at the cost of remote data processing and potential charges.

There is no moral prize for choosing one configuration over another. Choose deliberately. For private manuscripts, sensitive notes, and material you have no per-

mission to upload, a local model is usually the safer starting point. For demanding synthesis, a user may choose a remote service only when the source and provider terms make that acceptable. Omnitome's architecture gives you this choice; it cannot make the choice on your behalf.

PART II

Getting the First Session Right

4

First Launch: Start Small, Learn the Room

A freshly installed reading application can be intimidating if the first screen is a blank library and an unfamiliar collection of model settings. Omnitome's v1.0.2 onboarding behavior tries to make that first encounter explicit. When neither a configured OpenRouter key nor a local LLM endpoint is detected, the application routes a new user to Settings with an onboarding banner. It explains that AI configuration is not yet complete and offers a **"Skip for now (Explore offline)"** path. That lets you look around, inspect existing content, or use supported non-generation paths without first solving model configuration.

Use this first session as an orientation, not as a benchmark contest. Confirm that the app opens reliably. Visit Settings. Look at the available provider fields. Explore the Library and Research surfaces if sample or imported materials are present. Inspect the Import tab and its queue. Check whether the local model resources listed by the first-run resource manager are installed. If resource setup is incomplete, let the application finish or resolve the download rather than launching several large model loads at once.

4.1 The four things to understand first

One: the Library is the working shelf. It is where derived Omni readings live and where you normally open a book or continue reading. It supports search and filtering so the collection remains navigable as it grows.

Two: Research is the evidence layer. It holds source-oriented material, including source dossiers and standalone research sources. It is not a competing "raw mode" toggle that changes the definition of the book. It is where you inspect or manage the source content that backs an Omni reading.

Three: Import is asynchronous work. Adding a file generally enqueues a task. Extraction, indexing, classification, outline planning, synopsis generation, synthesis, and provenance attribution can take minutes. While work runs, the queue reports its state. Closing the app during active work is not a good troubleshooting shortcut; let the job complete, or cancel it through the supported control if you need to stop it.

Four: Settings controls a real resource boundary. A model dropdown is not the same thing as a running model. The provider selected in Omnitome needs a valid endpoint, a compatible model identifier, and any required key. Neural

models such as BGE-M3 and Kokoro have their own local resource requirements, separate from the LLM selected for synthesis.

4.2 Your first import should be small

Before feeding Omnitome a 500,000-word textbook, a scanned archive, or a directory of books, test the entire path with a modest text file or a clearly structured public-domain source you have permission to use. Check that the title and basic text are plausible. Watch the queue progress through its stages. Open the resulting book. Confirm that an Omni reading exists if synthesis completed, and follow at least one provenance citation into Research. This tells you far more than a green success toast.

A small test makes problems legible. If it fails at the first stage, the issue may be format or extraction. If extraction works but the queue stalls while indexing, the neural sidecar or system resources may be involved. If chapter planning looks wrong, inspect the extracted headings and source structure. If the Omni text exists but its citations point somewhere unexpected, the issue is not the same as a failed file upload. The stages are valuable because they help you stop treating the app as one enormous black box.

4.3 Do not confuse resource setup with content import

Omnitome has two different kinds of “download” or “load” activity. The resource manager obtains required neural assets such as BGE-M3, reranking, OCR, and TTS models. The Import workflow processes your books and articles. Model setup prepares the machinery; import processes the material. A first-run resource failure can prevent local processing even though the file itself is perfectly valid. Conversely, a document can ingest successfully while AI synthesis is not yet configured, leaving source material available without a completed Omni reading.

That separation is useful rather than awkward. It means the app can report an incomplete model resource without pretending your book is corrupt, and it can preserve source-oriented material even when a synthesis step has to be retried. As you become familiar with the app, learn to ask the right question: *Did the file fail to parse, did the local neural resource fail to load, or did the LLM provider fail to synthesize?* The fix for each is different.

5

Choose Your Intelligence Provider

Omnitome's LLM settings are designed around two primary provider choices: **OpenRouter** and **Local**. The Local option includes presets for **Ollama**, **LM Studio**, and a custom OpenAI-compatible endpoint. The local pathway is not Ollama-only; the shared compatibility contract allows different local servers to present a familiar API surface.

A practical configuration has four ingredients: the provider type, its base URL, a model identifier, and an API key if the provider requires one. OpenRouter uses a known remote base URL and a provider-issued key. A local server uses a loopback address such as `http://localhost:11434/v1` or `http://localhost:1234/v1`, the identifier of a model currently available in that server, and any key required by the endpoint. In the Omnitome Settings UI, the provider selector determines which set of fields is active.

5.1 How to choose

Choose **OpenRouter** if you want a hosted language model accessed through a single API gateway, and if the text you process may be sent to that remote service under terms you have reviewed. This can reduce the burden of downloading, storing, and running a large generation model locally. It may also create usage-based costs. Check your provider dashboard and configured limits before processing a large corpus.

Choose **Ollama** if you want a local server with a relatively direct model-pull and run workflow. The default compatibility base URL in Omnitome is `http://localhost:11434/v1`, and the example model field is `llama3`. That is a preset, not a guarantee that the model is already installed or that this exact tag is available on your setup. The model name you select must match the one exposed by your running Ollama server.

Choose **LM Studio** if you prefer to manage local models through its desktop interface and use its Developer tab to serve an OpenAI-compatible endpoint. Omnitome's preset base URL is `http://localhost:1234/v1`. The model field is intentionally left blank by the preset because the exact loaded model identifier depends on what you installed and loaded in LM Studio.

Choose **Custom** when you have another local or privately hosted endpoint that speaks the API shape Omnitome expects. Custom does not mean "any endpoint on the internet." It means you are responsible for supplying a compatible service, valid base URL, reachable route, and supported model identifier. A service that

merely happens to produce text is not necessarily compatible with the client's request format.

5.2 Test first, save second

The Settings screen provides a **Test Connection** control. Use it after entering the provider information. A successful test reports connection status and latency; a failed test returns a message that can help narrow down the issue. A successful test proves that the test request could reach and use the configured provider at that moment. It does not prove that every future long-context synthesis will fit the model's limits or that the model will preserve every nuance of a particular novel or paper.

After testing, press **Save Settings**. The provider selection used for future synthesis is stored through the app's settings service. Avoid assuming that changing a field visually changed the active provider before settings were saved. If you switch between Local and OpenRouter, test the new choice and save the change before starting a large queue.

5.3 Privacy and cost are part of configuration

An OpenRouter API key is a credential. Do not paste it into a public issue, share it in screenshots, or commit it to a repository. Create and manage the key in your provider account, and set a spending limit where the provider offers that control. The generation model and the local embedding model are separate things: selecting a remote LLM does not mean BGE-M3 embeddings are necessarily being generated remotely, and selecting local synthesis does not mean all local neural dependencies have already been installed.

With a local server, the absence of a remote model invoice does not mean the task is costless. It uses disk space, RAM, GPU memory, electricity, and time. Smaller models are easier to run but may be less capable on complex synthesis. Larger models may improve instruction following but require more memory and may process long documents more slowly. Start with one representative book, measure the quality of the resulting chapter units, and only then decide whether to change the model.

6

Connect OpenRouter

OpenRouter provides a single API surface for a range of hosted language models. Omnitome's settings code uses the base URL `https://openrouter.ai/api/v1` and sends the selected model identifier along with the key. The public repository's Settings screen includes preset choices, but provider catalogues evolve. Treat those presets as convenient starting points and confirm that the chosen identifier is currently available to your account.

6.1 Step 1: Create a key outside Omnitome

Sign in to your OpenRouter account using the official website. Create an API key from the account's key-management area. Give it a label that helps you recognize what it is used for, and consider a spending limit or expiration policy if those controls are available. Copy the key when it is displayed and store it carefully. Secret keys are credentials; a screenshot, chat transcript, log, or repository commit can turn a private key into a public one.

6.2 Step 2: Open the provider settings

Launch Omnitome and open **Settings**. In the AI provider section, select **OpenRouter**. The interface should show an API-key field and a model field. The current code uses `deepseek/deepseek-v4-flash` as its initial value and lists several presets, including DeepSeek V4 Flash, Claude 3.5 Sonnet, and Llama 3.3 70B. Those exact catalogue entries can become stale as providers retire or rename models; check the live provider listing or choose an identifier you know your account can use.

6.3 Step 3: Enter the key and model

Paste the key into the API-key field. Keep it masked except when you have a clear reason to reveal it. Choose a model appropriate for long-form synthesis, not simply the model with the most appealing marketing name. Important qualities include instruction following, long-context reliability, ability to retain qualifications, consistent response formatting, throughput, and price. A model that returns a fast generic summary may be less useful than one that preserves the source's structure and caveats.

Omnitome's OpenRouter base URL is managed by the application, so the ordinary user does not need to paste a different URL for the standard OpenRouter service.

If a future version exposes additional routing controls, follow that version's field labels rather than copying a remembered configuration from this edition.

6.4 Step 4: Test the connection

Select **Test Connection**. A successful result should report a connected state and latency in milliseconds. If the test fails, do not immediately start a book import and hope the error goes away. Check for an incomplete key, stray whitespace, an unavailable model ID, an exhausted spending limit, provider-side service trouble, or a network restriction. Re-enter the key from the provider dashboard if necessary, then run the test again.

6.5 Step 5: Save and make a small proving run

Click **Save Settings**. Import a small, well-structured text or document that you have the right to process. Let the queue finish, then examine the synopsis, outline, and a few synthesized chapters. Follow citations into Research. Quality is the real connection test. A provider can accept requests and still produce output that is too vague, too verbose, poorly structured, or difficult to attribute for your intended material.

6.6 What travels to the provider?

When OpenRouter is the selected generation provider, treat the content supplied to the synthesis request as leaving your computer. Depending on the operation, this may include source excerpts and instructions derived from the imported book. Review the provider's current policies before sending confidential, personal, unpublished, restricted, or licensed content. Do not interpret "local-first" as a promise that OpenRouter requests are processed on your device.

6.7 Usage and cost discipline

Long books can generate many requests across synopsis, outline, synthesis, retry, or re-synthesis tasks. Costs can accumulate even when each request seems inexpensive. Set appropriate account-level limits, monitor usage, and test with a small work before enqueuing a large collection. If you frequently re-synthesize the same book while tuning prompts or models, treat those iterations as experiments with real costs.

A practical model-selection routine is to import one representative chapter or a smaller work, review the results against its source, and record what went wrong. Did the synthesis collapse a counterargument? Did it invent a connective detail? Did it omit a critical definition? Did citation chips lead to useful source passages? Then change one variable at a time: model, source structure, or import quality. This is more informative than comparing models by one generic chat prompt.

7

Connect a Local LLM with Ollama

Ollama runs language models on your computer and exposes an OpenAI-compatible API at `http://localhost:11434/v1` for applications that use that request format. Omnitome's Local provider includes an Ollama preset with that base URL and an example model name of `llama3`. The model's actual tag must match the one installed locally and exposed by your server.

7.1 Step 1: Install Ollama and obtain a model

Install Ollama from its official distribution for your operating system. Start the local service if it is not already running. Download a model that fits the resources available on your machine. A commonly used command is:

```
ollama pull llama3
```

This command is an example aligned with Omnitome's preset. Model tags change and different variants have different memory requirements, so check the actual model tag you chose. You can inspect your installed models with:

```
ollama list
```

Do not download a model merely because its name is familiar. A high-parameter model can consume substantial system memory or GPU memory; a quantized variant may be more practical. The best choice is the model that fits your hardware and can reliably handle Omnitome's synthesis workload.

7.2 Step 2: Confirm the server and model

Ensure the Ollama application or service is running. From a terminal, you may check the available model list via the CLI. For a network-level check, Ollama's OpenAI-compatible endpoint supports a model list route at `http://localhost:11434/v1/models`. The exact output depends on installed versions and local configuration. If the route cannot be reached, solve that before opening Omnitome's test connection.

For local inference, keep the service bound to the local machine unless you understand and intentionally configure network access. Exposing a model server to a wider network changes the security picture. An application endpoint intended for loopback use should not casually be made reachable from untrusted devices.

7.3 Step 3: Select Local in Omnitome

Open **Settings** and choose **Local** as the provider type. Set the preset to **Ollama**. The app fills the base URL with:

```
http://localhost:11434/v1
```

It also starts with `llama3` as an example model name. If you installed a different model or tag, replace that value with the exact identifier shown by Ollama. The `/v1` suffix matters for the OpenAI-compatible API. Do not substitute Ollama's native `/api` endpoint unless a future Omnitome version explicitly supports that API shape.

7.4 Step 4: Handle the local API key field

A local Ollama server generally does not authenticate the same way as a hosted service. However, OpenAI-compatible clients may require the API key field to contain a non-empty string even if the local server ignores it. The Ollama compatibility documentation uses a placeholder such as `ollama` for this reason. Omnitome's local key field is provided for endpoint compatibility and custom servers; whether it is required depends on the configured route and the application version. If the connection test says the key is missing, use the placeholder accepted by the local-compatible client. Do not reuse your OpenRouter secret for a local endpoint.

7.5 Step 5: Test, save, and prove quality

Run **Test Connection**. If it succeeds, click **Save Settings**. Then import a short source. A working local model may still behave differently from a hosted frontier model. Watch for loss of qualifications, repeated generic language, malformed chapter boundaries, refusal to handle long input, or output outside the expected structure. Try a modestly sized local model first, then adjust only when the source-based result gives you a clear reason.

7.6 Hardware and context expectations

Local generation draws on RAM, GPU memory, compute throughput, and the model's context window. Omnitome's ingestion pipeline also runs separate local neural operations for embedding, reranking, and possibly OCR. These systems may take turns using a limited GPU, and the project has previously had to guard against a four-gigabyte graphics card being overwhelmed at startup. A machine that can comfortably run one model interactively may become constrained when several local components are warmed at the same time.

That is why lazy loading and resource management matter. If generation becomes slow or a local model stops responding, close unrelated GPU-heavy applications, confirm that the model server is alive, and try a small task. Avoid interpreting high utilization by itself as a deadlock. At the same time, do not ignore a process that no longer makes progress. Check the actual queue state and logs where available, and preserve the source so that you can retry safely after resolving the resource problem.

8

Connect a Local LLM with LM Studio

LM Studio is another local-model route. It gives users a desktop application for discovering, downloading, loading, and serving models. Its OpenAI-compatible server commonly uses port `1234`; Omnitome's preset base URL is `http://localhost:1234/v1`. The Local provider supports this alongside Ollama and a custom endpoint.

8.1 Step 1: Download and load a model in LM Studio

Install LM Studio from its official distribution. Use its model interface to download a model suitable for your computer. Load that model in the Chat or Developer area. It is important to distinguish a model that exists in the download cache from one that is currently loaded and available to answer requests. A model can be installed but not serving.

8.2 Step 2: Start the local server

Open LM Studio's **Developer** tab and enable the server control. In the documented default configuration, the server listens at port `1234`. Some LM Studio versions also support starting the server with the `lms server start` CLI command. Check the server display for the actual address and port; if you changed them, use the values LM Studio reports rather than assuming the defaults still apply.

8.3 Step 3: Find the model identifier

Use the model identifier reported by the LM Studio server, not a nickname you invented for your library. Compatible APIs include a `/v1/models` route. The exact value may resemble a model repository name or the loaded model's configured identifier. Copy the value shown by the server or documentation for your version.

8.4 Step 4: Configure Omnitome

In Omnitome, open **Settings**, choose **Local**, and select the **LM Studio** preset. The expected base URL is:

```
http://localhost:1234/v1
```

Enter the actual model identifier in the model field. The preset leaves that field blank to avoid pretending that every user has loaded the same model. If your server is configured to require an API token, enter the token it expects in Omnitome's local API-key field. If authentication is disabled for the loopback server, do not add a hosted provider key as a substitute for a local server setting.

8.5 Step 5: Test and save

Run **Test Connection** and wait for the result. If the server is not reachable, check that LM Studio remains open, the model is loaded, the server is started, and the host and port match. If the server responds but rejects the model, copy the model ID from the running server. If the request connects but synthesis quality disappoints, try a model better suited to long structured generation rather than assuming the API endpoint is broken.

After the test succeeds, select **Save Settings**, then run a small import through the complete pipeline. Verify that Omni chapters appear, and inspect their provenance. A simple "hello" response proves that the server can answer a small request; it does not prove that it can preserve nuance across a book-length workflow. The source-first verification routine remains the correct quality check.

8.6 Custom endpoint: the same discipline

When using another OpenAI-compatible endpoint, enter its base URL and exact model identifier only after checking that it supports the request route used by Omnitome. Some services implement only a subset of the OpenAI API, have custom authentication rules, or require separate enablement for chat completions. A URL that looks plausible is not evidence of compatibility. Confirm endpoint support, test connection, save the configuration, and verify with a short but representative document.

PART III

From Source to Reading

9

Import Is a Workflow, Not a File Picker

The Import surface is the front door to Omnitome's processing pipeline. It is deliberately broader than a file selector because reading material does not arrive in one shape. You may have a book stored on disk, a paper on the web, a public-domain text copied from a source, or a research article that should be added to an existing dossier. Omnitome's Import route is designed to make those entry points part of one observable queue.

The route has four tabs: **File Upload**, **Web Discovery**, **Paste Text**, and **Import Queue**. The chosen tab can be reflected in the URL, so an import method may be opened directly or revisited without mentally reconstructing the last choice. The tabs are keyboard navigable as well as clickable. This little detail matters in a working tool: repeated imports should be predictable, and controls should remain operable without a mouse-only ritual.

9.1 File Upload

Use File Upload when the source is already on your device. The upload surface supports selecting or dropping one or more files. Multi-file selection is useful for a small batch of papers or books, but it should not be mistaken for a promise that every item will complete at the same speed. Large PDFs, scanned documents, multilingual works, and EPUBs with unusual structure can demand very different amounts of extraction and analysis.

After selecting files, the app queues tasks in the background and takes you to the queue view. You can continue navigating the application while processing runs. The persistent import dock is intended to surface current work without monopolizing the screen. If a task finishes, the interface can offer an **Open book** action to take you directly to the completed result.

Before importing a large batch, check the current import size ceiling in Settings and make sure you have sufficient disk space. Omnitome processes more than the original file alone: extracted text, chapter structures, semantic chunks, embeddings, generated content, cache files, and database records can add their own footprint. The exact multiplier depends on file type and processing decisions. A small first run lets you learn the resource profile of your system.

9.2 Web Discovery

Use Web Discovery when the source is an article or accessible URL rather than a local file. Omnitome's documented web path uses a readability-oriented parser and metadata extractors for selected source families, including academic and public-domain pages. The Import interface treats web imports as jobs rather than forcing potentially slow retrieval and parsing to block the entire page.

A URL is not automatically a complete source. It may point to a login wall, a dynamic page whose content is rendered in a way the extractor cannot access, an anti-bot interstitial, a page with only an abstract, or a document that is not textually accessible. A successful download may also omit footnotes, complex tables, embedded images, equations, or supplementary files. When the page matters, compare the extracted text with the original web page before relying on the resulting Omni reading.

For research, prefer the primary source where practical. If you are importing a summary of a study, make sure the title and publisher metadata describe the summary rather than the study itself. Provenance can connect synthesized claims to the imported source; it cannot turn a secondary report into primary evidence.

9.3 Paste Text

Paste Text is useful for short notes, excerpts, plain-text chapters, or material you are authorized to process but that does not exist as a ready-to-import file. Pasted content enters the same broad ingestion path, which means it can be normalized, indexed, classified, and synthesized like other sources.

The source you paste should have enough structure for the intended task. A paragraph can be helpful as a research note, but it may not yield meaningful chapter boundaries. A multi-chapter text with headings gives the editorial planner better clues. If the text was copied from a PDF or legacy webpage, check for broken line wraps, missing punctuation, unusual spacing, or garbled characters before import. Paste can preserve the text you provide; it cannot reconstruct characters that never made it onto the clipboard.

Omnitome uses SHA-256 content checks for incoming files and text pastes to detect exact duplicates. When a match is found, the interface presents a replace-or-cancel decision. This is a guard against accidentally creating two entries for the same material. It is not a semantic duplicate detector: two different files that contain almost the same prose, or an updated edition with a small change, may not be treated as identical by a cryptographic exact-content check. Consider the prompt carefully before replacing an existing record.

9.4 Import Queue

The queue tab is not just a history list. It is the visible task manager for work that may continue after you leave the Import page. It reports the state of tasks and provides actions such as cancel and retry where supported. It is the first place to go when you wonder whether a file is still processing, whether an import failed, or whether the application simply needs time to finish a large stage.

9.5 One source, several identities

A source can contribute to more than one useful workflow. A standalone research article can remain a source record. A set of related documents can be treated as a dossier. A book can have a source representation in Research and a derived Omni reading in the Library. The fact that one object can have connected views is not a reason to merge them conceptually. Keep asking which role a record currently plays: evidence, derived reading, or both.

10

The Queue: Watch the Work Happen

A long import can look deceptively simple from the user's point of view: choose a file, click a button, wait for a book. Internally, Omnitome has to transform a file or URL into a structured source, split and normalize text, build a semantic index, infer content type, plan chapters, synthesize the reading units, and attribute their sentences to source passages. The queue makes those stages explicit enough that a wait has a reason and a failure has a likely location.

The current handoff records a staged progress map with these landmarks:

Stage	Progress landmark	What it broadly means
Ingest	15%	Acquire and extract the source into usable text and metadata.
Semantic indexing	30%	Prepare chunks and vectors that help retrieve relevant source passages.
Classification	45%	Determine the broad content type and related descriptive signals.
Outline	55%	Plan the editorial structure and chapter-like units.
Synopsis	60%	Produce a higher-level account that can orient the reading experience.
Synthesis	70%	Generate compact reading units from the planned source material.
Attribution	85%	Resolve source passages for the derived statements.

Stage	Progress landmark	What it broadly means
Done	100%	Mark the task complete.

These percentages are progress landmarks, not a stopwatch. A stage with heavier work can take much longer than the stage before it. The team specifically corrected a bug where progress appeared stuck at 35% during semantic indexing; the current reported behavior uses monotonic stage weights and normalizes progress within the active stage. Progress should move forward rather than dropping arbitrarily or suggesting that the pipeline has finished before attribution is complete.

10.1 What happens in the pipeline

Ingest. The application accepts the source and routes it through the appropriate parser. For a PDF, that may mean layout-aware extraction and OCR fallback for pages with no extractable characters. For an EPUB, it means walking the publication's structured content. For a URL, it may mean retrieving the accessible page and using a readability parser. Normalization converts diverse source formats into an internal textual shape that later stages can work with.

Semantic indexing. The system divides the text into chunks suitable for vectorization and later retrieval. BGE-M3 computes embeddings locally; the project records a 1024-dimensional embedding path with FP16 CUDA acceleration where supported. A cross-encoder reranker can refine candidate passages. These systems do not author the reading by themselves. They help the pipeline identify which sections of a long source are most relevant to a chapter or a generated statement.

Classification and outline planning. The book is categorized and an outline is built from document structure, headings, and other signals. This stage is especially important for long novels and technical works. If a source's table of contents, heading typography, or chapter markers are extracted incorrectly, the resulting structure may be awkward even if the underlying words are present. Inspect the outline when the chapter list looks suspicious.

Synopsis and synthesis. The synopsis offers a broad orientation. The synthesis stage generates the smaller reading units. This is where the selected LLM provider is most directly involved. The project has built range checks and retries to reduce malformed or oversized output; those controls are intended to keep chapter units readable and consistent, not to certify that every sentence is perfect.

Attribution. The system resolves the synthesized sentences to source material. The purpose is to give the reader a route from the compressed prose back to the evidence. Attribution is a separate stage because generation and source mapping are separate tasks. A generated chapter can exist before citation resolution is complete, but the finished workflow aims to reconcile them.

10.2 Why tasks can take different amounts of time

A small text file may pass through the pipeline quickly. A 500,000-word textbook requires more parsing, indexing, outline planning, synthesis requests, and attribution. A scanned PDF can spend additional time in OCR. A multilingual source may need encoding detection and specialized segmentation. A local LLM depends on its own inference throughput, while OpenRouter depends on network latency, provider response time, model rate limits, and your account. A GPU that is already using memory can slow the embedding and reranking stages.

Do not interpret every long pause as failure. Look at the active stage, whether progress or logs are changing, and whether the local resource or provider is responding. The correct question is not "Why isn't the bar filling instantly?" but "Which work is active, and is there evidence it is still progressing?"

10.3 Retry and cancellation

Use cancel when you deliberately need to stop a job, not as a repeated attempt to wake it up. Canceling may leave you with a failed or incomplete task that needs retry, and stopping the application abruptly may interrupt state transitions. Use the queue's explicit retry action after resolving the likely cause. For example, reconnect a stopped local LLM server before retrying synthesis; install missing OCR or embedding resources before retrying parsing or indexing; fix an inaccessible source URL before retrying web extraction.

The queue's task records and database lifecycle reconciliation also exist to prevent content from getting stranded between stages. Recent work added reconciliation hooks so books whose chapter synthesis completes are marked as Library items, even if an earlier lifecycle row was created before chapters existed. If an imported work does not appear in the Library immediately, first check queue status and refresh the collection. Do not immediately re-import the same file, since duplicate detection and concurrent jobs can create confusion rather than fix the cause.

11

Formats, Extraction, and the Limits of Parsing

Omnitome accepts a wide range of long-form material. Its documented ingestion engine includes PDF, EPUB, DOCX, RTF, Markdown, raw text and paste, and accessible web URLs. It also contains explicit handling for multilingual sources and legacy encodings. That breadth is useful, but "supported" means that a parser path exists and has been exercised against test fixtures. It does not mean every file with that extension will yield perfect text or that every feature in its original layout will survive extraction.

11.1 PDF: text, layout, and scans

PDF is a page-description format, not always a semantic document format. Some PDFs contain selectable text with useful structure. Others are scans made of page images. Many combine text, figures, tables, footnotes, two-column layouts, and unusual font encodings.

Omnitome uses a layout-aware PyMuPDF path for text-based PDFs and can invoke PaddleOCR when pages yield no extractable characters. The source records also show work on mathematical glyph normalization and experiments with math extraction that were later gated off after over-detection. The practical takeaway is important: support for PDF equations is not a promise that every equation, symbol, or multi-column table will be reconstructed perfectly. Math-heavy papers, OCR pages, and diagrams deserve manual spot-checking.

Before importing a large PDF, open it in a normal PDF viewer and test whether the text is selectable. If selection copies sensible prose, the text path may work well. If copying returns nothing, a few garbled symbols, or words in a strange order, expect OCR or layout challenges. For OCR-sensitive documents, compare the extracted source against a few pages containing headings, punctuation, formulas, or tables. If the extraction is wrong, synthesis can only build on that imperfect input.

11.2 EPUB: structure can help, but metadata can still be messy

EPUB's internal XHTML/XML spine often gives a cleaner semantic structure than a visually complex PDF. Omnitome's parser extracts the publication's content blocks. However, EPUBs may contain front matter, repeated navigation pages, advertise-

ments, publisher notices, or chapter titles embedded in unusual markup. A table of contents can leak into extracted text if structural filtering is imperfect. The project history includes explicit fixes for TOC filtering, chapter title artifacts, and heading ancestry.

If the resulting chapter list seems to begin with a contents page or contains duplicated headings, inspect the source representation rather than assuming the synthesis model invented the structure. A title artifact may originate in parsing. For novels, source quality and chapter boundaries often matter more than fancy metadata.

11.3 DOCX and RTF: text with inherited formatting

DOCX ingestion uses `python-docx` to extract text and preserve structural headings where available. RTF ingestion strips legacy formatting through a dedicated parser. Both paths are useful for notes, reports, and office documents, but complicated documents may rely on text boxes, embedded objects, headers and footers, or layout conventions that a simple text extraction path cannot fully preserve. Check tables and cross-references when they carry meaning.

11.4 Markdown and plain text

Markdown supports headings and block structure that can make outline planning substantially more reliable. Omnitome uses a CommonMark-oriented parser with special handling for certain text shapes such as dialogue segmentation. Plain text is more ambiguous because chapter boundaries may be indicated only by blank lines, repeated punctuation, or inconsistent headings. Before import, use clean headings where appropriate, preserve paragraph breaks, and remove obvious copy artifacts.

Pasted text follows a UTF-8-oriented path with a buffer sniffer and mojibake correction. If the original clipboard already lost punctuation or substituted replacement characters, there may be no reliable way to reconstruct the missing text automatically. Inspect a representative portion before asking the AI to compress it.

11.5 Web articles and URLs

The web import path uses readability extraction and metadata helpers for selected source families. Dynamic pages, paywalls, authentication gates, robots restrictions, or unusual front ends may prevent full extraction. A page may load in your browser while returning little useful article text to an automated parser. Confirm that the extracted source includes the sections you care about. If a primary paper's full text is available as a PDF or another accessible document, importing that source may yield a more complete record than importing a landing page.

11.6 Languages and character encodings

The project includes a Tier 1 fast path for clean UTF-8 and a Tier 2 statistical fallback for legacy encodings such as Shift-JIS, Windows-1252, GB18030, and other

non-UTF-8 text forms. It contains special handling for Japanese Aozora works, Western European accents, Russian encodings, Chinese character counting, and CJK-aware sentence boundaries. These are substantive features because broken encoding can destroy names, punctuation, and sentence segmentation before synthesis even begins.

Still, language support should not be interpreted as equal performance across every language or writing style. Test a sample that represents the actual text you intend to read, especially if it uses older encodings, uncommon punctuation, verse, mixed scripts, or specialized terminology. For a Japanese work, check chapter headings and sentence boundaries. For Chinese text, check word counts and segmentation. For mathematical or technical text, inspect symbols and equations. If source extraction is defective, repair the source before spending time tuning the LLM.

11.7 A simple source-quality protocol

Before committing to a long import, ask four questions:

1. Is the source complete, legal for my intended use, and readable in its original viewer?
2. Does the extracted text preserve titles, paragraphs, punctuation, and the order of the content?
3. Does the outline represent actual sections rather than repeated table-of-contents entries or layout artifacts?
4. Do a few generated statements map to useful source passages?

This protocol prevents a common category mistake: blaming the synthesis model for a failure caused by extraction, or blaming the parser for a sentence that was simply synthesized poorly. A good reading environment treats each stage as its own source of truth and its own possible failure.

PART IV

Your Working Library

12

The Library: A Shelf That Remembers

The Library is the home of your derived reading collection. It is where you return to books you have imported and where you can search, filter, open, continue, and manage the work. It is not a generic file directory. A Library entry can represent a completed Omni reading, a source that has not yet been synthesized, or a book with a lifecycle state that determines whether its source dossier or derived content is present. The interface uses status indicators such as **Source Only** or **Source detached** to make those distinctions less mysterious.

At the top of the Library, you will find the collection count and a primary **Add Book** action that takes you to Import. The search field matches title, author, and description. Type filters distinguish all books, novels, web/articles, and documents according to the available metadata. Tag filters allow you to narrow the collection by classification signals, and the selected tag and search filters are persisted across navigation. The top eight tags by frequency may be surfaced as useful shortcuts.

12.1 Search like a librarian, not like a database administrator

Begin with the most specific signal you know. Search the title if you remember it. Search the author if titles are similar. Use a word from the description if the title is generic. Then use content type or tags to reduce the results. Clear filters when the collection looks unexpectedly empty; because filters can persist between visits, the "missing" book may simply be hidden by a previous combination of search terms and tags.

Tag filters use an **AND** combination in the current Library filtering logic: if you select several tags, an entry must match every selected tag to remain in the result list. This is stricter than an OR search. For example, selecting "machine learning" and "textbook" narrows to works that have both tags rather than every item that has either tag. If an expected title disappears, inspect the selected tags and clear filters before searching again.

Classification tags are helpful hints, not an ontological truth. A classifier can assign an imperfect category, and a novel may have meaningful crossover with history, politics, or technology. If a work fails to appear under the category you expected, try "All Books" and direct search rather than treating the classification as a gatekeeper.

12.2 Open a book

Clicking a book card opens the book's details and chapter list. The visual cover is generated procedurally from the title and theme rather than necessarily being an uploaded cover image. That gives the collection a consistent visual rhythm and lets the design communicate states such as uncompressed source-only items. It does not mean the cover artwork was authored by the source publisher.

The details area acts as the bridge between the collection and the reader. It shows metadata, summary context, and chapter units. Large works benefit from chapter filtering and ordering support, so the project added a search input that matches chapter title or sequence/index. Use this instead of scrolling through dozens of chapter cards when returning to a specific section of a long textbook or novel.

12.3 Select and batch-manage

The Library supports a selection mode for bulk deletion. Select the intended items, review the selected count, and use the destructive action only when you understand what representation you are removing. The bulk-delete flow synchronizes relevant queries and prunes associated storage records according to the operation. It is convenient for cleaning up duplicate imports and abandoned test materials, but it should not replace deliberate source management. A Library removal is not always conceptually the same operation as deleting a research source, archiving a dossier, or removing the heavy original file while keeping the compact reading.

Before batch deletion, ask whether the items contain useful source material, whether you may want to re-synthesize them later, and whether their source files are stored elsewhere. If the source matters, keep an independent backup. A local database is not a substitute for a separate copy of irreplaceable files.

12.4 Reading progress and return paths

Omnitome includes reading-progress behavior designed to remember where a user left off. Progress can be informed by opening a chapter, scrolling toward the end, and using explicit reading controls. Those signals serve different purposes. Opening a chapter is an intentional start, reaching the end suggests completion, and a manual "mark read" action can express user judgment. Treat progress as a memory aid rather than a scholarly record of exactly how many words you attended to.

For a large collection, the Library is most useful when you maintain it as a living shelf. Remove duplicate tests when they no longer matter. Keep active reading material discoverable. Use tags and descriptions to support later retrieval. Keep source records where you expect to revisit the evidence. The goal is not to make every card immaculate; it is to reduce the friction between remembering a book and returning to the idea you were reading.

13

Book Details and the Chapter Map

A book's detail page answers a different question from the Library grid. The grid asks, "Which work should I open?" The details page asks, "What is inside this work, and where should I begin?" For long-form reading, that second question is essential. A 500,000-word book with thirty or more chapters cannot depend on one long scroll and a reader's memory alone.

Omnitome builds an editorial outline from the imported source structure and then generates compact chapter reading units. The details page exposes the chapter list, supports filtering by chapter title or sequence, and offers actions to read or continue, synthesize, or revisit the relevant unit when the interface provides them. The exact controls available depend on the state of the book: a completed reading has different useful actions from a source-only entry whose synthesis has not been generated.

13.1 Why the outline deserves attention

Chapter boundaries are not a cosmetic detail. In a novel, the chapter is often a narrative unit with a reveal, a shift in perspective, a scene change, or an emotional turn. In a textbook, it may introduce a new concept and assume knowledge from the previous one. In a research paper, headings such as Methods, Results, and Limitations define rhetorical roles. If the outline merges or fragments those units incorrectly, the compression stage may start from a bad editorial map.

Look for obvious warning signs: chapter titles that are duplicated, a table of contents listed as a real chapter, a heading that appears under the wrong parent section, an opening "chapter" containing publisher metadata, or a long source reduced to an implausibly small number of sections. The project's history contains specific fixes for title deduplication, table-of-contents filtering, heading ancestry, and CJK/Aozora chapter detection. That work reflects a simple truth: good synthesis starts with sensible structure.

If the outline is wrong, inspect the Research source. Check whether the parser extracted the headings in order. If the source structure is genuinely inconsistent, you may need a better source version or a cleaner text input. Re-synthesis is useful when the existing structure and source text are sound but the derived chapter is poor; it cannot reliably fix a heading that the import process never captured.

13.2 Synopsis is a map, not a verdict

The synopsis gives an overview of the work and can help you decide whether to read linearly, jump to a section, or revisit a previous concept. It should not be treated as a replacement for either the source or the chapter-level reading. Project work explicitly separated synopsis generation from outline generation so a synopsis fallback could not be confused with a reliable chapter plan. This distinction matters: a short descriptive paragraph can be a useful orientation even when the chapter structure remains incomplete, but the two operations do different jobs.

If the synopsis says only that a work "explores themes of power and identity" while the book is a technical manual, inspect the source and generated output. The issue may be classification, model behavior, or a fallback response triggered by insufficient input. A fluent synopsis is not a diagnostic instrument by itself.

13.3 Re-synthesis: use it with a hypothesis

Omnitome includes a **Re-synthesize all** path for cases where chapter synthesis was interrupted or earlier output fell back to a heuristic representation. Use it when you have a reason to believe the synthesis can improve: you have fixed the LLM endpoint, changed a model that was producing poor structure, corrected source extraction, or resolved a timeout. Re-synthesis may re-run generation and attribution work, and with a hosted model may incur more usage.

Avoid repeated re-synthesis without examining what went wrong. If the same broken source text is passed into the same model with the same constraints, the result may vary without becoming better. A disciplined retry changes one likely cause at a time and inspects the source afterwards.

13.4 A useful quality pass

After the outline and first generated chapters appear, perform a three-point check:

- **Coverage:** Does the chapter unit cover the main point of the source section, including important counterarguments or exceptions?
- **Pacing:** Does it read as a coherent unit rather than a flattened list of unrelated statements?
- **Traceability:** Do the citations lead to passages that genuinely support the sentence, rather than merely sharing a topic word?

This small review is more useful than judging output by word count alone. The range contract helps make the reading experience predictable, but a concise chapter that misses the argument is not a successful compression. Likewise, a longer unit is not automatically better if it repeats material without adding meaning.

14

Read in Omni, Verify in Research

The central reading rhythm in Omnitome is straightforward: **read the derived text for momentum, open the source when precision matters**. Omni Reading is the active, compact layer where you can move through the work at a higher information density. Research is the supporting source layer, where you can inspect the evidence behind the compressed statement.

When you open a chapter from the Library, you enter the Omni reading experience. The reader provides chapter navigation, a reading canvas, and controls for actions such as narration and fullscreen where available. Formula rendering uses KaTeX for supported mathematical expressions. The view may be styled by the current theme, and the relevant source citations are presented as part of the reading flow.

14.1 The first pass

For your first pass through a chapter, resist the urge to investigate every line immediately. Read the synthesized unit as a coherent piece. Notice its key claims, new concepts, causal links, decisions, or narrative events. If it appears to omit a central qualification, makes an abrupt jump, or says something you cannot reconcile with the surrounding context, mark that moment mentally or use the nearby citation to investigate it. Compression should accelerate orientation, not make you surrender curiosity.

14.2 The verification pass

Select a citation associated with the sentence you want to verify. Omnitome's provenance path is designed to navigate to the corresponding material in Research and highlight the source passage. A successful jump gives you a concrete place to inspect, rather than forcing you to find the paragraph by guessing. In multi-source work, the supporting evidence may come from a particular imported source within a dossier rather than one monolithic document.

Read enough surrounding context to judge the relationship. A source paragraph can support a claim only with its neighbors: the next paragraph may add a limitation, a prior sentence may define a term, or a footnote may qualify the result. Citation links get you to the neighborhood. Your reading supplies the judgment.

14.3 When to stay in Omni

A compact reading is especially useful when you want to survey a large work, review a chapter's central argument, revisit an already understood section, or listen while doing something else. It can be a reasonable first pass through an unfamiliar field, a memory refresh before an exam, or a way to find the chapter that deserves a full close reading. It is not necessary to switch to Research after every sentence just because the system exposes citations.

14.4 When to return to the source

Return to the source when you need an exact quote, original wording, a theorem or equation, precise numerical values, a surprising or contested claim, an exception to a general rule, or the sequence of evidence used to support an argument. For fiction, verify a consequential plot turn or a characterization that feels flattened. For legal or policy texts, consult the full relevant passage and authoritative version before acting. For academic work, read the methods, definitions, limitations, and context rather than citing a generated paragraph as if it were the author's words.

14.5 Fullscreen and chapter navigation

Omnitome supports both windowed and fullscreen reading. You can use the F11 key or the visible fullscreen controls. The state is synchronized with the browser/webview fullscreen API, and Esc can exit fullscreen while updating the interface state. Fullscreen is particularly useful for a distraction-free session, while the normal window is more convenient when comparing notes or checking a source alongside the reader.

When a book has many chapters, use the chapter list and chapter-search field on the details page rather than repeatedly stepping through sections. When switching chapters, the reader can reset scrolling to the top of the new unit, and the Research view supports independent scrolling panes for navigation and the main reading canvas. Those small navigation contracts matter because reading long material often involves revisiting one specific section many times.

The point of this workflow is not to make the source a second-class citizen. Omni gets you moving; Research lets you look under the hood. A mature reader uses both without confusing either for the whole truth.

15

Citations, Provenance, and Trust You Can Inspect

"Trust the AI" is too weak a design goal for a reading application. It asks the user to decide whether fluent language deserves belief. Omnitome instead tries to make trust an inspectable relationship between a statement and its supporting source material. In this context, **provenance** means the record of where a generated statement came from and how a reader can return to the supporting source passage.

The project began with provenance as a backend contract and steadily moved it into the visible reader. The implementation history covers a provenance resolver, source attribution, sentence segmentation, source chunk relationships, multi-source attribution, visible citation chips, anchored previews, and scroll restoration. Those were not ornamental features. They were necessary to turn a claim such as "the model says this section argues X" into a claim the reader can interrogate.

15.1 What a citation is doing

A provenance marker links a sentence or segment in Omni Reading to one or more source passages. The system uses semantic vectors to identify relevant material and attribution logic to connect the derived representation to source blocks. The visible control is the user-facing end of that process. Selecting it should open the relevant source content and highlight the supporting passage so you can inspect the underlying text.

Some generated sentences may have support spread across multiple source passages. A careful attribution system must allow more than one supporting chunk when the source's argument is distributed across paragraphs. The development records describe multi-source provenance and the move away from forcing every sentence to one source chunk. The reader should therefore treat a citation as a pathway to relevant evidence, not as a claim that the sentence is a direct quotation from one paragraph.

15.2 What provenance proves

A functioning provenance path proves something valuable but narrower than "the model is right." It shows that the system has linked the generated statement to

source material that can be inspected. If the link is meaningful, you can assess whether the source actually supports the claim, whether the model omitted an important qualifier, or whether several pieces of evidence have been combined into a reasonable inference.

Provenance also separates two forms of error. The synthesis can misstate a source even when the citation points to the right place. Or the synthesis can be broadly accurate while the attribution points to a passage that is too general or merely topically related. The first is primarily a content problem; the second is a grounding or attribution problem. The practical remedy is to inspect both the statement and the passage, not to assume every visible chip has the same evidentiary strength.

15.3 Why deterministic linking matters

A citation that points somewhere unpredictable is frustrating and weakens trust. A durable provenance system needs stable IDs and consistent representation boundaries. The development history includes fixes for mismatched representation selection, real block IDs, ancestor resolution for nested headings, and alignment between semantic chunks and visible source blocks. These problems show why citations are not simply prompt formatting. They require the parser, data model, source viewer, and frontend navigation to agree about what a source passage is.

A deterministic route is one that resolves to an identifiable source block rather than a fresh, free-form answer each time you click. It lets the interface open the same evidence context repeatedly and makes the relationship debuggable. If a source was appended later to a dossier, its provenance should resolve to the correct source item, not just to a vaguely related paragraph elsewhere in the collection.

15.4 Citation does not mean quotation

Omni Reading is generated prose. Even when its citations are correct, its wording is not necessarily the author's wording. Do not copy a generated sentence inside quotation marks and attribute it directly to the original author. If you need a quotation, locate the original sentence in Research or in the original file, verify punctuation and context, and quote that source under the rules applicable to your use.

Similarly, do not cite an Omni synthesis as though it were the primary scientific evidence, legislation, or source text. Use it as a reading aid. For consequential work, open and cite the original material, and check dates, version, methodology, definitions, and limitations.

15.5 Citation quality checklist

When a citation matters, ask:

1. Does the highlighted passage actually contain the claim's essential facts or reasoning?
2. Has the generated sentence removed a condition, exception, timeframe, or un-

certainty?
3. Does the source passage depend on nearby text that changes its meaning?
4. Is the claim supported by this source alone, or does it combine evidence from several passages?
5. Is the citation to the original source that you intended to use, especially in a multi-source dossier?

When the answer is uncertain, expand the surrounding source context, not just the selected line. If you find a mismatch, treat it as useful quality feedback: record what was wrong, inspect the import and outline, and consider a targeted re-synthesis after you identify the cause.

15.6 Absence of a citation is also information

A missing or unavailable citation means that the statement is harder to verify through the intended reader path. It should not be treated as a signal of greater model confidence. Depending on the record's state, source may be detached, attribution may not have completed, or an item may be a source-only entry without an Omni reading. If a compact reading persists after source detachment, the interface should indicate that condition rather than make the source appear to remain available.

Trust grows when the application shows its limits. The most useful outcome is not a reader who believes every generated sentence. It is a reader who knows when to keep moving, when to click, and how to inspect the evidence with very little friction.

16

Research: Dossiers, Sources, and Context

Research is the source workbench behind the Library reading experience. It is where imported source material is organized, where dossiers can gather related documents, and where a reader can inspect the original extracted content that supports Omni sentences. The product evolved toward a clear principle: Smart Reading is the main reading product; Research is the evidence layer that backs it.

A **standalone source** is a source document or article that exists as a research item without necessarily having a generated Library reading. A **dossier** is a source-oriented container that can gather multiple related materials under a common book or research identity. A dossier might begin with one paper and later receive supplementary articles, notes, commentary, or another source that provides context. The append-source pipeline is designed to let those additions be retained with a timeline record rather than treated as invisible changes to the original source.

16.1 Why use a dossier?

Use a dossier when one question spans multiple sources. For example, you might import a foundational paper, then append a technical explanation, a critique, and your own research notes. Keeping those materials together makes it possible to consult them as one related research object while preserving distinctions about where a passage originated. The source provenance system must still identify the specific source block that supports a generated claim.

Do not create a dossier merely because two documents share a keyword. They should have a meaningful relationship. The quality of any multi-source synthesis depends on the relationship among the materials, the completeness of their extraction, and the ability to distinguish agreement from disagreement. Two papers with opposing results should not be flattened into a single smooth assertion that hides the conflict.

16.2 Read the source with context

The Research viewer supports a source-oriented reading canvas and chapter navigation, with independent scrolling for the left navigation pane and the main reading area. When you arrive from a citation, Omnitome aims to jump to the appropriate source passage and highlight it. You can read above and below the highlight to

understand scope, assumptions, and exceptions. A highlight identifies the target; it does not tell you which interpretation is correct.

When researching technical material, pay special attention to definitions, methods, tables, equation assumptions, and limitation sections. A model may correctly compress a paper's headline result yet understate the context needed to interpret it. In literary works, inspect the original scene around a condensed plot description. Dialogue, sequencing, and subtext can be essential even if they do not dominate a summary-style representation.

16.3 Append sources carefully

When a dossier accepts an appended source, the new item should be treated as an addition to the evidence collection, not a silent rewrite of the original. Check its title and metadata. Confirm that the extracted article contains the full sections you intended to add. If you later synthesize or use a multi-source reading, remember that the output can draw connections across sources, and verify which one supports each claim.

Use the append log or visible source list where provided to understand the dossier's history. The existence of an append log is important because research collections evolve: you may add a source after an earlier reading has been produced. When exact provenance matters, note whether the Omni representation predates or postdates the source addition and whether the derived text has been re-synthesized to account for the new material.

16.4 Delete sources only after deciding what should remain

The Research surface includes destructive deletion actions for dossiers or standalone source records, with confirmation and removal of associated semantic chunks. Deleting research evidence is different from removing only the Library representation. Before confirming, decide whether the source should remain as a reference, whether a generated reading depends on it, and whether a local source file exists elsewhere.

Research is a workspace, not a promise of infinite retention. Source data can be removed. The application's lifecycle rules distinguish what is active in Library, what exists in Research, what is archived, and what has been purged. A careful user treats that as a feature for deliberate data stewardship rather than a surprising implementation detail.

16.5 Research is not an external fact-checker

Omnitome can help you check a synthesis against the documents you supplied. It does not automatically validate the truth of every source, detect every conflict across the wider literature, or prove that a source is authoritative. A perfectly traceable statement can still reflect a poor source. For medical, legal, financial, safety-critical, or current factual decisions, consult authoritative primary sources and appropriate professionals where necessary. Research makes the provenance of the imported corpus visible; it does not magically turn that corpus into a com-

plete picture of the world.

PART V

Reading With Your Ears and Senses

17

Local Neural Narration with Kokoro

Omnitome's reading experience includes a local neural text-to-speech engine based on **Kokoro-82M**. The purpose is not simply to turn a paragraph into a sound file. Narration should be able to accompany long-form reading: speak chapter titles, follow the sentence currently being read, advance through nearby material with minimal gaps, and let the user move through a book without constantly operating the transport controls.

Unlike an ordinary hosted text-to-speech service, the Omnitome voice path is designed to synthesize locally. The Python sidecar handles the neural work; the Node service coordinates requests and cache behavior; and the frontend plays audio and updates the visible reading position. The packaged v1.0.2 runtime includes the phonemizer data and spaCy model assets required by Kokoro's dependencies so that voice initialization does not unexpectedly try to download missing packages at runtime.

17.1 Voices and speed

The curated voice set recorded in the v1.0.2 application includes four identifiers:

Voice ID	Display name	Character described in the project documentation
`af_heart`	Heart	Warm, gentle American female voice
`am_adam`	Adam	Grounded, authoritative American male voice
`bf_alice`	Alice	Crisp, articulate British female voice
`bm_george`	George	Resonant, literary British male voice

Settings also exposes speed choices from 0.75× to 2.0×. Begin near 1.0× for technical or unfamiliar material. Increase speed for a review pass only when comprehension remains comfortable. A fast narration that causes you to miss definitions, names, or causal links defeats the purpose of a denser reading format.

The voice and language of the source may not be a perfect match. The current curated voices are described as American- or British-English voices; do not assume they are an ideal pronunciation engine for every language, fictional naming system, mathematical notation, or acronym. When reading unfamiliar names, listen for likely pronunciation errors and return to the text if the spoken form is ambiguous.

17.2 Gapless playback and sentence highlighting

The frontend uses a dual-deck playback design that can prepare adjacent sentence audio ahead of the playhead. It decodes WAV buffers into the Web Audio API's native audio buffers and uses a shared `AudioContext` across UI sound effects and narration. This design addresses an important desktop-webview detail: asynchronous audio playback can be blocked if the audio system was never unlocked by a user gesture. The reader's play and resume interactions explicitly resume the shared context.

During playback, the current sentence can be highlighted and the viewport can auto-scroll with it. The auto-scroll behavior is intended to remain considerate: when you manually scroll away from the playhead, the reader should yield rather than forcibly dragging you back. The user remains in charge of the reading canvas.

Kokoro can also narrate a chapter title before reading the chapter body and support chapter-to-chapter auto-advance, sometimes described as podcast-style progression. This turns a single play button into a longer listening session. Before leaving it running, ensure that the selected book is the one you intend to hear and that the output volume is comfortable.

17.3 Why a citation can become silence

Some synthesized text segments may contain only citation backlinks or punctuation. Those segments have no meaningful speech content after sanitization. The TTS integration handles this by returning a very short silence buffer rather than throwing an exception that breaks the playback queue. This is an implementation detail with a user-visible purpose: a non-spoken citation marker should not destroy the continuity of narration.

17.4 Cache, startup, and performance

Narrated audio is cached on disk using content-addressed paths and an LRU strategy, with the Settings screen recording a default cache limit of 500 MB. Replaying a sentence may be faster when its audio already exists in the cache. The cache can be cleared from Settings when you need to recover space or refresh generated audio. Clearing the cache does not mean the source text itself has been deleted.

The Python sidecar uses lazy loading and lifecycle behavior to keep heavy neural models from consuming resources indefinitely. The project documents idle eviction of Kokoro weights after several minutes of inactivity, releasing video memory for other tasks. A first play after eviction may need a model reload and feel slower than a subsequent sentence. This can be normal. If audio never begins,

however, check whether narration is enabled, whether the sidecar reports readiness, whether the selected voice is available, and whether the system has sufficient memory.

17.5 A good listening routine

Open a chapter and read its heading before pressing Play. Start at normal speed for a new subject. Let sentence highlighting orient your eyes, but do not treat it as a substitute for looking at formulas, tables, or source citations that need visual interpretation. Pause before an important claim and inspect its provenance if necessary. Use auto-advance for material that benefits from continuous listening, and turn it off when you want to reflect at chapter boundaries.

Listening is particularly useful for reviewing familiar material, keeping a narrative moving during a walk, or reducing the visual burden of a long reading session. It is less ideal as the only mode for a proof-heavy textbook or a paper whose meaning depends on equations and diagrams. The best reading system offers more than one path; the best reader knows when to switch.

18

Five Themes, Five Reading Moods

Omnitome's theme system is built around the idea that the reading environment can have a personality without making the source material less authoritative. The application includes five visual themes: **Omni, Sakura, Spring, Coffee, and Cyberpunk**. Each gives the interface a distinct palette and accent behavior, while the Original Reading remains governed by the principle that source content should feel like a neutral reference.

18.1 Omni: the signature library

Omni is the product's signature identity: deep violet, cosmic atmosphere, and glowing purple motifs. It suggests that a reading session is an active passage through ideas rather than a plain file viewer. The theme is especially tied to the emblem and visual language that distinguish the product.

18.2 Sakura: the quiet bloom

Sakura uses gentle cherry-blossom pink accents and refined editorial typography. Its effect is softer and lighter, well suited to narrative reading, reflective essays, or a session where the interface should feel calm rather than technical. The theme is not meant to turn every page into an illustration; its job is to give the environment a consistent mood while keeping text legible.

18.3 Spring: the green room

Spring uses sage-green tones, organic contrast, and generous whitespace. Green accents create a restrained visual rhythm that sits comfortably behind long sessions. Use it when you want the application to feel open and uncluttered.

18.4 Coffee: the study table

Coffee brings warm sepia and amber hues inspired by paper, leather bindings, and old libraries. It gives reading a familiar physical texture without pretending to reproduce a particular book or publisher's design. For readers who prefer warm backgrounds to stark white surfaces, this theme makes the interface feel more like a well-used desk.

18.5 Cyberpunk: the console at midnight

Cyberpunk uses high-contrast dark surfaces with neon cyan and magenta accents. It is the most overtly technological of the five. It can be appealing for late sessions or for readers who prefer dark mode, but ensure that the contrast and glow remain comfortable for your eyes over time.

18.6 What theme changes, and what it should not change

Themes affect the interface palette, accent colors, theme-aware components, scrollbars, and decorative treatment. The theme system also contributes to procedural book covers, which are generated from title-related information rather than copied from the source publisher. Switching themes may play a signature sound motif if theme-switch audio is enabled.

A theme should not change the factual meaning of a source citation, the original source text, or the identity of a model provider. It is presentation, not evidence. In particular, do not assume that a purple citation is more trustworthy than a pink one. The important properties are whether the citation resolves to the intended source and whether the highlighted text supports the claim.

18.7 Choosing a theme for the task

There is no universally correct theme. Choose the one that makes the text easy for you to read on your display and in your environment. A dark, high-contrast theme may feel comfortable in a dim room but less suitable in daylight. A warm theme may feel gentle but reduce readability if your display is poorly calibrated. If you change themes often, check that dialogs, settings controls, citations, and the source canvas remain clear, not just the Library cards.

Theme customization is not a substitute for healthy reading habits. Take breaks, adjust your display brightness, use the fullscreen mode when it helps, and disable sound effects when you need silence. The ideal theme disappears into the background when your attention turns to the words.

19

The Procedural Sound Engine

Omnitome has an offline UI sound system designed to give interactions a little tactile confirmation without relying on a remote audio service or synthesizing every sound at runtime. In v1.0.2, the repository describes **90 pre-rendered WAV assets**: eighteen canonical interaction events across the five themes. The files are PCM16, 48 kHz, mono audio, normalized to a -3 dBFS peak target. A deterministic Node.js sound-design pipeline generates the assets using procedural synthesis techniques rather than shipping a collection of unrelated stock sounds.

The five sound palettes mirror the visual themes but have their own identities. The project documentation names the audio palettes Classic, Paper, Terminal, Soft, and Mechanical; the visual theme names are Omni, Sakura, Spring, Coffee, and Cyberpunk. This is an intentional distinction in the sound-design vocabulary, not an inconsistency in the five-theme visual system. The runtime associates theme-specific sound assets with the active visual theme through the event map.

- **Classic** uses warm marimba and organic mallet textures.
- **Paper** evokes page turns, parchment slides, and tactile card movements.
- **Terminal** uses retro CRT blips, square-wave chirps, and telemetry-like tones.
- **Soft** favours gentle sine hums, ambient chimes, and smooth transients.
- **Mechanical** suggests switches, latches, and weighted spring detents.

The sound engine covers common semantic events: primary or secondary button actions, navigation between Library, Research, and Import, opening and closing a book, operation start and completion, warnings, toggles, and the signature motif that can accompany a theme change. These cues are intended to confirm state, not to compete with reading.

19.1 Why pre-render sounds?

Procedural generation is performed during development or asset preparation. At runtime, the player loads pre-rendered buffers into memory and plays them through the Web Audio API. This avoids the unpredictable cost of synthesizing a sound each time a button is clicked. It also allows the sound set to be tested as a known collection: asset count, waveform format, and peak level can be verified before release.

The application also uses a shared audio context with speech synthesis. This was part of the v1.0.2 work to resolve cases where the packaged WebView2 environment blocked narration despite the backend successfully returning a WAV file.

Operational chimes are ducked or suppressed while Kokoro narration is active, reducing the chance that a navigation sound will interrupt a spoken sentence.

19.2 The controls you can use

Settings exposes the global UI sound toggle, a volume slider, a separate toggle for theme-switch sounds, and a **Preview Motif** action to audition the active theme's signature cue. The implementation uses a perceptual volume curve rather than mapping slider percentages naively to raw waveform amplitude, so small slider changes can behave more naturally at quiet levels.

If you prefer silence, disable UI sound effects. You do not need to accept every designed sound to use the reader. During focused writing, study, or accessibility-sensitive use, silence may be the right choice. If you use TTS, keep in mind that the narration engine and UI sound effects have distinct controls. You can want one without the other.

Sound is a small part of an interface, but it carries information. An operation-start cue can reassure you that an action registered; a completion cue can signal that a transition finished; a theme motif can help a new visual environment feel deliberate. Good sound design is not the volume of the metaphor. It is the precision of the feedback.

PART VI

Stewardship and Practical Wisdom

20

Deletion, Detachment, and the Shape of Ownership

A reading library grows through use, and eventually you will need to remove something. The danger is that the ordinary word “delete” can hide several different operations. Omnitome’s lifecycle model distinguishes whether a derived Library reading is active, whether a source dossier is retained in Research, whether a record is archived, and whether the underlying source has been detached. That distinction is essential because the original material and the generated reading do not serve the same purpose.

The documented lifecycle model describes several meaningful outcomes:

Outcome	What remains	When it is useful
Remove the derived reading from Library	The Research source remains available; the Omni reading is removed from active Library use.	You want to stop seeing the synthesized item but preserve evidence for later work.
Archive both	The item is no longer active in Library or Research, but is retained as archived state until purged.	You want to set the work aside without treating the action as immediate permanent deletion.
Delete or detach the heavy source only	The lighter Omni reading may remain with a **Source detached** indicator.	You need to free source storage and accept that some provenance paths may no longer reach the full source.
Terminal purge	The archived lifecycle record and associated chunks are removed from storage.	You have deliberately decided that the material should be permanently removed.

The exact controls and confirmation language depend on the app version and the page from which you act. Do not infer that every row in this conceptual table is represented by a single button in every screen. The key principle is that removal from Library, removal from Research, source detachment, archive, and permanent purge are different lifecycle decisions.

20.1 Before you delete

Ask yourself what you may want to recover later. If the book is a primary reference, keep the original file outside Omnitome in a location you back up. If an Omni reading took hours to synthesize and you may want to revise it later, consider whether preserving the source will be worth the disk space. If you imported a duplicate, compare titles and metadata before removing one record. If the source is copyrighted or confidential, think about whether local retention is permitted or prudent.

The Library's batch deletion is useful for housekeeping, but bulk operations multiply the consequences of a misclick. Select a small number of entries, inspect the selected count, and review the confirmation dialog. A good habit is to clean up experimental imports one at a time until you understand what that version of the interface removes.

20.2 Source detached is not source preserved

A detached state means that a compact derived reading is still available while the full source is no longer attached in the expected way. It is not a mysterious storage optimization that makes the original evidence recoverable by magic. Citation markers may remain visible, but if the source material has been removed, a reader may no longer be able to inspect the full passage. The badge exists to communicate that limit.

When you need robust long-term provenance, keep the source. When you knowingly choose detachment to save disk space, record where the original file has been backed up and understand that verification may require restoring or reopening that file outside the current Library state.

20.3 Database, files, and backup

The local SQLite database contains structured records, lifecycle state, semantic chunks, chapter outputs, and related metadata. It is not necessarily the same thing as the original source file directory or the TTS audio cache. Backing up just the database may preserve parts of a reading collection without preserving every imported file or resource. Copying only the source files may preserve the evidence but not the generated Omni reading, notes, or progress. A meaningful backup plan should account for the components you care about.

For valuable material, prefer a documented backup routine over relying on the application's current state as the only copy. Close or safely stop active processing before making filesystem-level backups, and follow the project's current recovery instructions rather than copying a live database at an arbitrary moment. This guide cannot specify a universal storage path for every future release; consult the installed version's documentation and settings when building a backup plan.

21

Privacy, Copyright, and Responsible Use

Omnitome helps organize, compress, and listen to long-form content. That ability creates both convenience and responsibility. The user who controls an imported source also controls the decision to keep it, process it with local or hosted AI, export it, or share a derived reading with another person. The application can help preserve provenance, but it cannot decide the legal, ethical, or contractual status of every source.

21.1 Know the source's permission status

Before importing material, consider where it came from, what licence or terms apply, and what you intend to do with the result. Public-domain works are often suitable for experimentation, but status can vary by country and edition. A work made available on a website is not automatically in the public domain. A personal subscription or institutional login does not necessarily permit automated extraction or redistribution. A PDF you are allowed to read may still be restricted from uploading to a third-party provider or sharing its derivative text.

Compression is not a licence. Summaries and derived reading representations can still reproduce protected expression or reveal too much of the source, depending on content and jurisdiction. Do not publish a generated reading as an unofficial replacement for a commercial work without appropriate rights. For public demonstrations or starter libraries, prefer original writing you have the right to distribute or works whose legal status and source edition have been carefully verified.

21.2 Keep local and remote processing distinct

If you select Ollama or LM Studio with a local endpoint, text sent to that generation endpoint is intended to be processed on your machine. Verify that the endpoint URL is actually local and that you have not configured a remote server under a "local" label. Keep services bound to loopback unless you deliberately intend to expose them and have secured access appropriately.

If you select OpenRouter, the content needed for generation is sent to the remote provider. Review current data-use terms, retention settings, model-specific policies, and account configuration before processing confidential, personal, unpublished, or licensed material. A local embedding engine does not prevent a cloud

LLM from receiving the excerpts used for synthesis. Conversely, choosing a local LLM does not automatically mean the entire app has no network activity, because model downloads, web import, or update workflows may use network access when invoked.

21.3 Protect keys and private documents

Treat API keys as passwords. Do not place them in manuscript examples, screenshots, issue reports, or public repositories. Redact secrets from logs before sharing diagnostics. If you think a key has leaked, revoke it in the provider dashboard and create a replacement. Account-level spending limits help reduce financial surprises but do not remove the need to monitor usage.

For sensitive source documents, consider whether the device is shared, whether operating-system accounts are protected, where the app stores local data, and whether backups are encrypted. "Stored locally" is not the same as "inaccessible to anyone else." The security of a local-first tool includes the security of the device and user account on which it runs.

21.4 Be accurate when citing a derived reading

Use Omni Reading to orient yourself, explore, and remember. When you publish work, cite the original source as appropriate, inspect exact wording and relevant context, and disclose AI assistance where the policies or context require it. A provenance chip helps you find the source passage; it is not itself a complete bibliographic citation and does not guarantee that the source supports the full claim.

When using Omnitome for academic work, do not submit generated text as a substitute for your own analysis if your institution forbids it. When using it at work, follow your employer's rules on confidential documents and AI services. When reading fiction, respect the author's and publisher's rights rather than distributing a generated alternate edition. Responsible use preserves both the value of a tool and the ecosystem of material that makes it useful.

22

Troubleshooting Without Guesswork

Troubleshooting is faster when you classify the failure before changing settings. Omnitome contains several coordinated processes, a queue, model services, and a database. A problem that looks like “the book didn’t work” may be an extraction problem, a model-resource problem, a provider connection problem, a synthesis quality issue, a provenance navigation defect, or a UI cache issue.

22.1 The application opens, but onboarding returns to Settings

On first launch, the onboarding logic checks whether a cloud API key or local endpoint has been configured and whether the onboarding bypass has been dismissed. If no provider is configured, the app may send you to Settings. Either configure a provider and test it, or use **Skip for now (Explore offline)** if you only want to inspect the app without AI generation. If you already configured a provider but the prompt keeps returning, confirm that you saved the settings and that the current installation is reading the expected settings store.

22.2 Test Connection fails

For OpenRouter, verify the key, selected model identifier, network connectivity, and usage limits. For Ollama, confirm the local service is running, the `/v1` base URL is correct, and the installed model tag matches the configured name. For LM Studio, confirm that the Developer server is started, the selected model is loaded, and the port matches the server display. For a custom endpoint, check API compatibility, authentication, and whether the endpoint is reachable from the application process. Change one thing at a time, then test again.

22.3 A file imports, but the Library does not show it

Start in the Import Queue. Check whether the task is queued, active, complete, cancelled, or failed. A completed import can still require the app’s book list to refresh, particularly when state changes between a queue task and the Library query cache. The v1.0.2 record includes self-healing lifecycle hooks, startup reconciliation, and an event-based refresh path to address cases where smart chapters existed but a book was not marked active in Library. Refresh or navigate away

and back after checking the task. Do not immediately import the file again if the first job may still be running.

If the task failed, note which stage failed. Parsing errors call for a cleaner file or different source format. Missing model resources call for installing or downloading the required assets. A synthesis-stage failure may call for reconnecting the LLM provider and retrying. An attribution failure is different again: the source and Omni text may exist while provenance still needs attention.

22.4 Progress seems stuck

Look at the active stage, not just the percentage. Semantic indexing, OCR, local generation, and attribution can all take longer on big or difficult files. Verify that the task is still active and the relevant local service has not stopped. If there is no sign of progress and the machine is saturated, allow resources to settle, then consult the current support or diagnostic route. Avoid killing the application as a first response; the native supervisor is designed to clean up child processes on normal close, but forced shutdown may interrupt work in progress.

22.5 The chapter list looks wrong

Inspect the Research source and compare heading extraction. A table of contents may have been mistaken for content, a nested heading may be misplaced, or a scanned page may not have produced the expected text. Check for duplicated chapter titles, a title artifact at the start of the book, a chapter containing only front matter, or fragments that do not follow the source order. Re-synthesize only after fixing the source or structure that caused the problem.

22.6 The Omni reading is vague or inaccurate

Open the relevant citation and inspect the original paragraph with surrounding context. If the source itself is garbled or incomplete, improve the import first. If the source is sound but the synthesis omitted a necessary qualification, try a different suitable LLM, re-run the affected synthesis if the application supports it, and compare output against the same source. Avoid judging quality by prose smoothness. Record specific failure examples: omitted counterargument, invented causal link, wrong number, lost definition, or unsupported sentence.

22.7 Citations do not open the expected passage

Check whether the source is still attached and whether the item is a multi-source dossier. A compact reading may remain after source detachment, but the full source may no longer be available. For a source that is present, try the visible provenance action and inspect the Research source list. If the chip consistently resolves to the wrong source block, treat it as a provenance issue rather than a general synthesis issue. Preserve the book and source, record the exact sentence and destination, and use the current project support process to report the reproducible case.

22.8 Narration is silent

First confirm that TTS is enabled in Settings and that the reader's playback state is actually playing rather than paused. Click Play directly in the application to unlock the shared audio context. Confirm a voice is available, and check system output volume. Wait for the first local model warm-up if this is the first play after startup or idle eviction. Then try a different sentence or chapter. If the backend returns audio but the interface is silent, the problem may be in the front-end playback context; if synthesis fails, inspect sidecar readiness and missing model resources.

The v1.0.2 work specifically addressed WebView2 autoplay restrictions, PyTorch cuDNN library packaging, `espeak-ng-data`, and the spaCy `en_core_web_sm` model. These fixes make the packaged speech path more robust, but a corrupted installation or unsupported runtime environment can still cause issues. Do not attempt to fix a frozen runtime by manually installing random dependencies into the application's internal folder; use the official installer or current support instructions.

22.9 Narration stutters or a chapter skips

Try normal playback speed, pause and resume using the reader controls, and let the audio buffers initialize. The engine uses two decks and cached sentence audio for smoother transitions. Clearing the TTS cache may help if a cache is corrupted, but it should not be the first step if every voice is silent. If only citation-like segments are silent, that can be expected because citation-only text has no speech content; the pipeline uses a short silence buffer to preserve queue continuity.

22.10 The computer becomes slow or GPU memory fills

Omnitome's local neural components are substantial. BGE-M3, reranking, OCR, local LLM generation, and Kokoro can all compete for memory and compute. Close unrelated GPU-heavy apps, avoid warming every model simultaneously, and use the resource manager to see which assets are installed. The project adopted lazy loading and idle eviction precisely because a four-gigabyte GPU can thrash when several components are active. CPU fallback may be available for some paths, but it can be significantly slower.

If a first-run resource download fails, check disk space and connectivity and retry through the managed resource workflow. The downloader validates checksums and uses staging directories before promoting an archive to its final installed location. A failed or partial download should not be treated as a verified installed model. Avoid manually editing manifest locks or model directories unless following a project-maintainer procedure.

22.11 Sound effects are distracting

Open Settings. Disable UI sound effects globally, reduce the sound volume, or turn off theme-switch sounds while leaving narration enabled. Use the preview-motif control to understand what a theme change will sound like. Sound feedback is optional; it should never be a reason to abandon a comfortable reading session.

22.12 A general rule: preserve evidence, then repair

When a problem occurs, do not delete the source or repeatedly re-import it as an experiment. Preserve the source, note the failing stage, record the provider and model configuration without exposing keys, and reproduce the smallest case that demonstrates the issue. A small reproducible case often reveals whether the fault belongs to parsing, indexing, synthesis, attribution, or presentation. This discipline protects your work and makes any future support request far more useful.

23

Reading Recipes for Real Life

A user manual should not only explain the controls. It should help you develop a good working rhythm. Different sources call for different reading strategies, even when the underlying application is the same.

23.1 Recipe A: Survey a long technical book

Import the book in a format with clean text, preferably a text-based PDF or EPUB if both are available. Watch the queue through outline planning and synthesis. When the result appears, inspect the chapter map before reading. Search by chapter title when you know the topic you need. Read the Omni chapter for orientation, then follow citations for critical definitions, equations, exceptions, and claims you intend to rely on. Keep the source attached for the life of the project. A textbook is not just its conclusions; its examples and limitations are part of the learning.

23.2 Recipe B: Review a research question across sources

Create a dossier around a precise question rather than a broad keyword. Import the foundational paper, then append sources that add evidence, context, or critique. Keep an eye on disagreements: a good synthesis should not turn contradictory findings into false consensus. When a claim matters, open its citation and check which source supports it. If a newly appended paper materially changes the picture, decide whether the Omni reading needs to be refreshed to incorporate the additional evidence.

23.3 Recipe C: Read a novel without losing the story

Start with the synopsis to get an orientation, then read the Omni chapters in sequence. Treat the compact reading as a way to move through the narrative at a higher density, not as a demand to accept every condensation. When a relationship changes, a character makes a surprising decision, or a reveal seems too abrupt, use the citations to return to the source scene. If atmosphere, prose style, or dialogue is the point of the chapter, read the original in full. Compression is a second route through the story, not a verdict on what makes literature valuable.

23.4 Recipe D: Listen through a familiar work

Choose a theme and voice that make reading comfortable. Confirm the chapter title, press Play, and keep the reader visible if sentence highlighting helps you follow along. Use auto-advance for a continuous session, but pause at chapter boundaries if you want to retain notes. Increase speed gradually, not reflexively. For formulas, names, or exact claims, stop and inspect the text and provenance before continuing.

23.5 Recipe E: Test a new provider responsibly

Pick a short, representative source you are permitted to process. Import it with the current setup, then inspect the outline, synopsis, one early chapter, one middle chapter, and citations. Change only one configuration variable at a time. If switching from OpenRouter to Ollama, change provider and model settings, test the connection, save, and rerun the same source if appropriate. Compare not just writing quality but structure, coverage, citation support, speed, and resource use. The model that performs well on a greeting may not be the best model for long-form synthesis.

23.6 Recipe F: Prepare a source for a later project

Use Research as the durable evidence collection. Keep source titles and metadata clear. Append supplementary material rather than pasting it into an unrelated existing source. Keep a separate copy of the source files. If you decide to remove the Omni reading from the active Library, consider keeping the Research source so you can return to it later. If you delete or detach a source, make that a conscious storage decision rather than a side effect of tidying cards.

23.7 Recipe G: Recover from a messy import

First, inspect the source's text and headings. If it is a scanned PDF, compare OCR output against pages with dense text. If it is multilingual, check encoding and punctuation. If the chapter list contains repeated TOC entries, do not keep rerunning the LLM. Find out whether the wrong structure came from parsing. Repair the source or switch formats, then import again only when the new input is materially better. This is how you avoid paying for the same failure repeatedly.

These recipes share one principle: keep the source available, make one decision at a time, and use the application's visible state to guide the next move. Omnitome rewards curiosity and careful inspection more than blind button-pushing.

A

Quick-Start Checklists

A.1 A. First-session checklist

- ☐ Launch Omnitome and note whether the onboarding banner asks for an AI provider.
- ☐ Explore offline if that is all you need; configure a provider before expecting full AI synthesis.
- ☐ Confirm required local model resources are installed and there is enough disk space.
- ☐ Choose OpenRouter, Ollama, LM Studio, or a compatible custom endpoint.
- ☐ Test the connection and save settings.
- ☐ Import one small source you have permission to process.
- ☐ Watch the queue through completion instead of judging only by the initial toast.
- ☐ Open the resulting Library item and inspect the chapter outline.
- ☐ Open at least one provenance citation and verify the source passage.
- ☐ Test narration and sound controls if you plan to use them.

A.2 B. Before importing a very large book

- ☐ Confirm the source is complete and readable in its original application.
- ☐ Check the configured file-size ceiling and available disk space.
- ☐ Check that the chosen LLM provider can handle your intended workload.
- ☐ Ensure local services are running and suitable model assets are installed.
- ☐ Avoid starting several large imports simultaneously on a resource-constrained computer.
- ☐ Confirm the selected provider's privacy terms and likely usage cost.
- ☐ Plan to review the outline and citations before trusting the synthesis.

A.3 C. Before deleting or detaching a source

- ☐ Decide whether the Library reading, Research source, or both should remain.
- ☐ Confirm whether the source file is backed up elsewhere.
- ☐ Check selected item counts before any batch operation.
- ☐ Read the confirmation dialog carefully.
- ☐ Understand that source detachment can leave a reading without full provenance access.

☐ Keep important research materials in a separate backup location.

A.4 D. Before sharing a generated reading

☐ Confirm you have the right to share the source-derived output.
☐ Remove confidential text and private credentials.
☐ Verify important factual claims against the original source.
☐ Do not present generated wording as a direct quote from the original author.
☐ Follow institutional, workplace, publisher, and provider rules for AI-assisted work.

B

A Compact Glossary

Attribution — The processing stage that connects generated reading statements to relevant source passages.

BGE-M3 — A multilingual embedding model used locally to turn text chunks into vectors for semantic retrieval.

BGE Reranker v2 — A cross-encoder reranking component that can refine a list of candidate source passages.

Dossier — A source-oriented record that can group related research material and receive appended sources.

Embedding — A numerical vector representation of text that helps a system compare semantic relationships rather than relying only on exact keyword matches.

Import queue — The task manager that records and exposes the stages of importing and processing sources.

Kokoro-82M — The local neural text-to-speech engine used for narration in the packaged application.

LLM — Large language model. In Omnitome, this usually refers to the model used for synopsis or synthesis, selected through OpenRouter or a local compatible endpoint.

Local-first — An application design that keeps local data and local execution central while allowing explicitly configured remote services where needed.

Omni Reading — The derived, compact reading representation presented through the Library and reader.

Original Reading — The source-oriented representation associated with Research and used to verify derived statements.

OCR — Optical character recognition, used to recover text from page images when ordinary text extraction yields no usable characters.

OpenRouter — A hosted API gateway through which Omnitome can access selected remote language models.

Provenance — Information and navigation paths showing which source passages support a generated statement.

Research — The app surface used for source-oriented reading, dossiers, and evidence management.

Resource manager — The local model setup system that downloads, verifies, extracts, and installs required neural assets.

Semantic chunk — A source text unit used for indexing and retrieval, intended to preserve enough context for useful matching.

Smart Chapter — A development-era and internal term for a synthesized chapter-sized reading unit.

Source detached — A lifecycle state indicating that the compact reading may remain while its full source file or source record is no longer attached in the expected form.

Synthesis — The generation of compact reading prose from planned source material, with the goal of preserving essential meaning and structure.

Tauri — The native desktop application framework used by Omnitome's Rust shell.

Two-Representation Invariant — The design rule that Original Reading remains distinct from the derived Omni Reading, with provenance linking the two rather than collapsing them into one editable text.

C

Technical Reference and Official Provider Guides

The application-specific facts in this guide are grounded in the Omnitome repository and the session handoff current to 22 October 2026. For implementation detail, see the public repository's `README.md`, `docs/FEATURES.md`, `docs/ARCHITECTURE.md`, and `docs/SESSION_HANDOFF.md`. The handoff records the v1.0.2 release state and recent packaging, audio, onboarding, and test work. Consult the live repository for changes made after this edition.

For provider setup, use official documentation as the live reference because interfaces and model catalogues evolve:

- OpenRouter Developer Platform — API base URL, models, and getting started.
- OpenRouter: Create API Keys — creating and managing keys.
- Ollama API Introduction — local server basics and base URLs.
- Ollama OpenAI Compatibility — using an OpenAI-compatible client with a local Ollama model.
- LM Studio as a Local LLM API Server — starting the local server.
- LM Studio OpenAI-Compatible Endpoints — base URL and model identifier expectations.

These outside provider references explain how the underlying services work; they do not imply that every feature in those products is exposed by Omnitome. Use Omnitome's Settings controls as the authoritative description of the integration points supported by your installed version.

About the Author

The Omnitome Engineering Team

The Omnitome Engineering Team builds tools for people who want to read deeply without surrendering either their time or their ability to question the machine. Omnitome grew through successive experiments in text extraction, semantic indexing, long-form synthesis, citation provenance, local neural narration, desktop packaging, and interface craft. The team's working principle is that intelligent assistance should make source material easier to navigate, not harder to inspect.

This field guide was prepared for the Omnitome Synthetic Archive as a companion to the v1.0.2 product state documented in October 2026. It is intended to help readers begin carefully, configure the tools they actually use, and preserve the distinction at the heart of the application: the lens can be useful, but the source remains the evidence.

Omnitome Publishing
October 2026